

**SIMATS ENGINEERING
ASSESSMENT TOOL 2**

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 3

Duration: 1 Hour
Total Marks: 50

Annexure A

Scenario Based Assignment

Code of Conduct:

I Sanjanaa Sree.P (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Sanj

$20 + 20 + 23 + 20 + 13$
 $= 96$
100

Unit 2: Assessment Tool 2

Scenario-based Assignment

Course Code: CSA1710 Artificial
Intelligence

Name: Sanjanaa Sree P

Reg. No: 192311454

1). AI - Agent for Auction - Bidding Game:

(i) How Minimax models the Problem:

The auction - bidding game can be modeled as a two - player adversarial game where:

- (*) MAX Player: Our AI bidding agent.
- (*) MIN Player: Competing bidder trying to reduce our profit.
- (*) Each node represents a market state and current bid.
- (*) Branches represent possible bidding actions.
- (*) Leaf nodes represent final outcomes such as profit, winning probability or loss.

Minimax selects the bid that gives the best worst case outcome.

(ii) Computational Challenges:

The search tree is very large because there are many possible bids and multiple bidding rounds.

Time complexity: $[O(b^d)]$

where b is the branching factor and d is search depth.

This causes high computation and memory usage making real-time decisions difficult.

(iii) Alpha - Beta Pruning:

Alpha - Beta Pruning removes branches that cannot affect the final minimax decision.

(*) Alpha (α) = Best value currently available to

(*) Beta (β) = Best value currently available to

(iv) Evaluation Function:

$$E = w_1 P + w_2 W - w_3 R - w_4 C$$

where P : Expected Profit W : Winning Probability
 R : Financial Risk C : Bid cost.

(v) Recommended Strategy:

Use Alpha - Beta Minimax with depth-limited search, heuristic evaluation and iterative deepening. This provides near-optimal decisions while meeting strict time limits.

1. Autonomous Underwater Vehicle (AUV):

(i) Greedy Best-First Search:

Greedy Best-First Search chooses the node that appears closest to the survivor.

$$f(n) = h(n)$$

where $h(n)$ is the estimated distance to the survivor.

Strength: fast and uses less computation.

Weakness: Ignores actual travel cost, may select dangerous or blocked paths and does not guarantee the optimal route.

(ii) A* Search:

A* considers both the cost already travelled and the estimated remaining cost:

$$f(n) = g(n) + h(n)$$

$g(n)$ = actual travel cost

$h(n)$ = Estimated cost to the survivor.

It can account for currents, depth, energy use, and obstacles, giving a better balance between speed and cost.

(iii) Effect of Heuristic Accuracy:

An accurate heuristic reduces the number of nodes searched and improves speed.

(*) GBFS is highly dependent on the heuristic and can make poor choices if it is inaccurate.

(*) A* is most reliable because it also considers actual cost.

(iv) Dynamic Environment:

Changing currents, debris and visibility can make existing paths invalid.

Both algorithms may need to replan, increasing computation.

(v) Recommendation:

A* is preferred because it provides a better balance of speed, cost, Optimality and safety. Real-time replanning should be used when the environment changes.

cloud workload Balancing :

problem formulation

The goal is to minimize :

Energy + latency + Hardware / wear.

while satisfying server capacity and workload requirements. Traditional exhaustive search is inefficient because thousands of servers create a huge search space.

(i) Hill climbing :

Hill climbing starts with a workload distribution and repeatedly moves to a better neighboring distribution.

(ii) Limitations :

(*) Local Maxima : Gets stuck at a locally good solution.

(*) Plateaus : Many solutions have similar values, so there is no clear direction.

(*) Ridge : Improvement may require several changes together, which Hill climbing may miss.

(iii) Simulated Annealing / Genetic Algorithms :

Simulated Annealing sometimes accepts worse solutions, helping escape local optima.

Genetic Algorithms (GA) use selection, crossover and mutation to explore many solutions and find better workload distributions.

(iv) Recommendation :

Genetic Algorithms are suitable for large cloud systems because they can handle large search spaces, multiple objectives and changing workloads. For faster simpler optimization, Simulated Annealing is also a good choice.